



ĐẠI HỌC THỦY LỢI

THUYLOI UNIVERSITY

LÀM VIỆC VỚI FORM

Giảng viên: Kiều Tuấn Dũng, Nguyễn Tu Trung
BM HTTT, Khoa CNTT, Trường ĐH Thủy Lợi

Hà Nội, 2024

Nội dung

- ❖ Cơ bản về Form
- ❖ Session và Cookie
- ❖ Ví dụ minh họa:
 - ❖ Làm việc với TextBox, Button
 - ❖ Làm việc với SelectBox
 - ❖ Chú ý: SV tự ôn lại với các điều khiển khác
- ❖ PHP – Ajax

Cơ bản về Form

- ❖ Khái niệm
- ❖ Khai báo
- ❖ Danh sách các input
- ❖ Phương thức GET
- ❖ Phương thức POST
- ❖ Ví dụ phương thức GET và POST
- ❖ Biến \$_REQUEST
- ❖ Validate
- ❖ Submit
- ❖ Bảo mật form

Khái niệm Form

- ❖ Form là thành phần không thể thiếu trong các ứng dụng Web vì form là nơi trao đổi và thu thập dữ liệu từ người dùng
- ❖ Form có 2 loại chính:
 - ❖ Form lấy thông tin dưới dạng căn bản
 - ❖ Form lấy thông tin dưới dạng các file được upload lên
- ❖ Có thể không sử dụng form để lấy thông tin từ người dùng không ?
 - ❖ Có thể, sử dụng kỹ thuật Ajax
- ❖ Hai phương thức thường được sử dụng khi gửi dữ liệu từ form là GET và POST
- ❖ Server phân biệt data gửi lên dựa vào thuộc tính name của các thẻ input form

Khai báo Form

❖ Cú pháp khai báo HTML

```
<form action='<url-xử-lý>' method='<tên-method>' >  
    <các-thẻ-input-html>  
</form>
```

❖ Trong đó:

- ❖ form, action, method là các từ khóa
- ❖ <url-xử-lý>: url dùng để xử lý các dữ liệu được gửi lên từ form
- ❖ <tên-method>: phương thức truyền dữ liệu (POST/GET...)

Danh sách các input

Component	Sample	HTML
Text Entry		<code><input type="text"></code>
Password Entry		<code><input type="password"></code>
Checkbox	☐	<code><input type="checkbox"></code>
Radio Button	☐	<code><input type="radio"></code>
Submit Button	<input type="submit" value="Submit Data"/>	<code><input type="submit" value="Submit Data"></code>
Reset Button	<input type="reset" value="Reset"/>	<code><input type="reset" value="Reset"></code>
File Selection Entry	<input type="file"/> Choose File No file chosen	<code><input type="file"></code>
Text Area (multiple lines)	<input type="textarea"/>	<code><textarea cols="20" rows="2"></code>
Select Menu (pop-up)	<input type="select" value="Red"/>	<code><select size="1"></code>
Select Menu (scrolling)	<input type="select" value="Red"/> Blue Green	<code><select size="3"></code>

Phương thức GET

- ❖ Phương thức GET gửi dữ liệu thông qua URL trên trình duyệt
- ❖ Các biến truyền lên có format:
`<URL>?param1=value1¶m2=value2 ...`
- ❖ VD: <http://google.com.vn?name=Anh&age=18>
- ❖ Thường sử dụng cho truy vấn lấy dữ (GET) liệu từ server
- ❖ GET giới hạn độ dài chuỗi dữ liệu trên URL là 1024 ký tự
- ❖ PHP sử dụng biến `$_GET` lưu trữ toàn bộ dữ liệu gửi lên
thông qua phương thức GET
- ❖ Không nên sử dụng GET để gửi các thông tin nhạy cảm như password...
- ❖ Dữ liệu gửi lên chỉ tồn tại khi submit form, vì vậy cần kiểm tra bằng hàm `isset()` để chắc chắn rằng đã submit form

Phương thức POST

- ❖ Có tính bảo mật hơn GET, dữ liệu truyền đi không hiển thị trên trình duyệt
- ❖ Thường sử dụng cho các truy vấn thay đổi dữ liệu **trong database (INSERT, UPDATE, DELETE)**
- ❖ Không giới hạn độ dài dữ liệu như GET
- ❖ Tương tự như GET, PHP sử dụng biến `$_POST` được dùng để lưu **trữ toàn bộ dữ liệu gửi lên thông qua phương thức POST**

Ví dụ phương thức GET và POST

- ❖ Xem file phuongthucGET.php
- ❖ <http://localhost/vidu/phuongthucGET.php>
- ❖ Xem file phuongthucPOST.php
- ❖ <http://localhost/vidu/phuongthucPOST.php>

Biến \$_REQUEST

- ❖ Có thể được sử dụng để lấy dữ liệu gửi lên từ form của cả 2 phương thức GET và POST
- ❖ Chứa tất cả thông tin về các biến \$_GET, \$_POST và \$_COOKIE (\$_COOKIE là biến quản lý liên quan đến cookie trình duyệt)
- ❖ Thông thường trong form đã chỉ định cụ thể tên phương thức => Ưu tiên sử dụng biến \$_GET hoặc \$_POST thay vì \$_REQUEST
- ❖ Chú ý: SV tự tìm hiểu thêm (nếu cần)

Kiểm tra hợp lệ Validate

- ❖ Là bước kiểm soát dữ liệu gửi lên từ user nhằm tăng tính bảo mật và tránh việc mất thời gian xử lý các dữ liệu rác
- ❖ Là bước không thể thiếu khi xử lý form
- ❖ Các lỗi nhập từ người dùng phổ biến là dữ liệu trống và nhập sai định dạng
- ❖ Quá trình validate dữ liệu thường gồm 2 bước:
 - ❖ Lấy dữ liệu từ user khi submit form và kiểm tra xem dữ liệu là hợp lệ hay chưa
 - ❖ Nếu chưa hợp lệ, thông báo lỗi tới người dùng, đồng thời giữ nguyên các giá trị các trường mà user đã nhập đúng để tránh cho việc user phải nhập lại
- ❖ Có thể validate bằng Javascript, trên server có thể validate lại một lần nữa (nếu cần)

Submit

- ❖ Tùy thuộc vào phương thức gửi dữ liệu lên là gì (POST/GET) để gọi biến `$_POST`/`$_GET` tương ứng
- ❖ Server chỉ xử lý logic dữ liệu từ user sau khi đã hoàn thành bước validate dữ liệu
- ❖ Form có thể được submit bằng cách click input submit hoặc nhấn Enter

Bảo mật form

- ❖ Tổng quan về bảo mật form
- ❖ Tấn công XSS
- ❖ Tấn công CSRF
- ❖ Chú ý: SV tự tìm hiểu thêm (nếu cần) và xem thêm qua ví dụ

Tổng quan về bảo mật form

- ❖ Bảo mật là vấn đề cực kỳ quan trọng đối với các hệ thống website
- ❖ Vì form là thành phần chính của việc truyền dữ liệu từ client đến server nên thường là đích đến của các cuộc tấn công
- ❖ Hiện nay các framework đều hỗ trợ rất tốt việc chống các tấn công này
- ❖ Hai hình thức tấn công phổ biến trong form là
 - ❖ XSS (Cross-site scripting)
 - ❖ CSRF (Cross-site request forgery)

Tấn công XSS

- ❖ Kỹ thuật này được thực hiện bằng cách chèn các mã script độc hại vào trong code của bạn
- ❖ Cách phòng chống
 - ❖ Không bao giờ tin tưởng dữ liệu mà user nhập, do vậy cần validate dữ liệu trước khi lưu vào database
 - ❖ Mã hóa các ký tự đặc biệt mà user nhập trong form thành các thực thể HTML, sử dụng hàm htmlspecialchars()

Tấn công CSRF

- ❖ Kỹ thuật này giả mạo chính chủ sở hữu của hệ thống đó
- ❖ Ví dụ:
 - ❖ Hệ thống của bạn có link xóa user dạng delete-user?id=xxx
 - ❖ Hacker biết được link này và gửi 1 mail cho bạn với 1 image có src là link trên, với id cụ thể
 - ❖ Khi click vào ảnh đó đồng nghĩa bạn đã xóa user!
- ❖ Cách phòng chống: Tạo key (token) cho form, với các bước xử lý như sau:
 - ❖ Thêm thẻ input ẩn với giá trị key và thuộc tính là hidden
 - ❖ Lưu key này vào session
 - ❖ Mỗi lần submit form sẽ kiểm tra key này có hợp lệ hay không, chỉ xử lý khi key hợp lệ

Session và Cookie

- ❖ Session
- ❖ Cookie
- ❖ So sánh Session và Cookie

Session

- ❖ Khởi tạo: `session_start();`
- ❖ Thêm dữ liệu: `$_SESSION['name'] = value;`
- ❖ Lấy giá trị: `$_SESSION['name'];`
- ❖ Lưu ý: Trước khi lấy giá trị cần kiểm tra biến session tương ứng đã tồn tại hay chưa sử dụng lệnh `isset()`
- ❖ Xóa session
 - ❖ Xóa 1 phần tử cụ thể: `unset($_SESSION['name']);`
 - ❖ Xóa toàn bộ session trên hệ thống: `session_destroy();`

Cookie

- ❖ Thường được dùng để lưu các giá riêng của từng trang web cụ thể cho client
- ❖ Cookie không mất đi khi đóng ứng dụng, sự tồn tại của cookie phụ thuộc vào thời gian sống khi bạn set cho nó
- ❖ Cookie được lưu dưới trình duyệt của client
- ❖ Biến toàn cục trong PHP lưu trữ các thông tin về cookie là `$_COOKIE`, có kiểu mảng

Cookie

- ❖ Khởi tạo: `setcookie(name, value, expire, path, domain);`
 - ❖ `name`: tên cookie muốn tạo
 - ❖ `value`: giá trị cookie
 - ❖ `expire`: thời gian sống của cookie
 - ❖ `path`: đường dẫn lưu cookie, mặc định là '/'
 - ❖ `domain`: tên domain
- ❖ Lấy giá trị: `$_COOKIE['name'];`
- ❖ Cần sử dụng lệnh `isset()` trước khi lấy giá trị
- ❖ Xóa cookie: Sử dụng lại phương thức `setcookie` như lúc khởi tạo, nhưng set thời gian sống nhỏ hơn thời gian hiện tại

So sánh biến Session và Cookie

- ❖ Giống nhau:
 - ❖ Dùng để lưu trữ thông tin giữa client và hệ thống
 - ❖ Có thể được truy cập từ mọi nơi trên hệ thống
- ❖ Khác nhau:
 - ❖ Session có tính bảo mật hơn cookie
 - ❖ Cookie được lưu trên trình duyệt của client **trong khi session được lưu trên server**
 - ❖ Session sẽ bị mất khi đóng ứng dụng, **trong khi cookie thì không**

Làm việc với TextBox, Button

- ❖ Xem file login.php
- ❖ <http://localhost/viduPHP/login.php>

Làm việc với SelectBox

- ❖ Xem file selectbox.php
- ❖ <http://localhost/viduPHP/selectbox.php>

PHP – Ajax

- ❖ Vấn đề refresh trang
- ❖ Khái niệm Ajax
- ❖ Khai báo Ajax
- ❖ Ví dụ Ajax

Vấn đề refresh trang

- ❖ Refresh trang có thể làm dữ liệu hiện thời bị mất
- ❖ Demo lại trang phương thức GET...
- ❖ Giải pháp: Sử dụng Ajax

Khái niệm Ajax

- ❖ Ajax – Asynchronous Javascript and XML
- ❖ Tạo ra các website bất đồng bộ, load dữ liệu mà không cần load lại trang
- ❖ Giúp tăng tốc độ, sự linh hoạt, hướng đến trải nghiệm tốt nhất cho người dùng
- ❖ Nên sử dụng Ajax với thư viện jQuery hơn là Javascript thuần
- ❖ Các thư viện Javascript khác có thể xử lý tương tự như Ajax như Angular JS, Node JS, React JS...

Khai báo Ajax

- ❖ url: url xử lý request ajax
- ❖ method: phương thức truyền dữ liệu (get/post)
- ❖ data: danh sách các biến gửi lên, có dạng
`<name>: <value>`

```
$.ajax({  
  url: '<url>',  
  method: '<method>',  
  data: {  
    <name>: <value>  
  },  
  success: function (result) {  
    //xử lý kết quả trả về  
  }  
});
```

- ❖ success: function (result): trường hợp trả về dữ liệu thành công được lưu ở biến result

Ví dụ Ajax

- ❖ Demo lại trang phương thức GET
- ❖ Demo trang phương thức GET có sử dụng Ajax
- ❖ Nhận xét: Có thể sử dụng Ajax để truy vấn CSDL

